Tutorial 4: IntelliJ, Java and Oracle

CSCI3170 Introduction to Database Systems

Learning Objectives

By the end of this tutorial, you should be able to:

1. Create a Java project in IntelliJ IDEA, add the Oracle JDBC driver, and connect to Oracle.
2. Perform CRUD with JDBC `Connection`, `Statement`, `PreparedStatement`, and `ResultSet`.
3. Explain transactions, `commit`, `rollback`, and try-with-resources.
4. Complete the two applications left from Tutorial 3: project budget adjustment and engineer workload rewards.

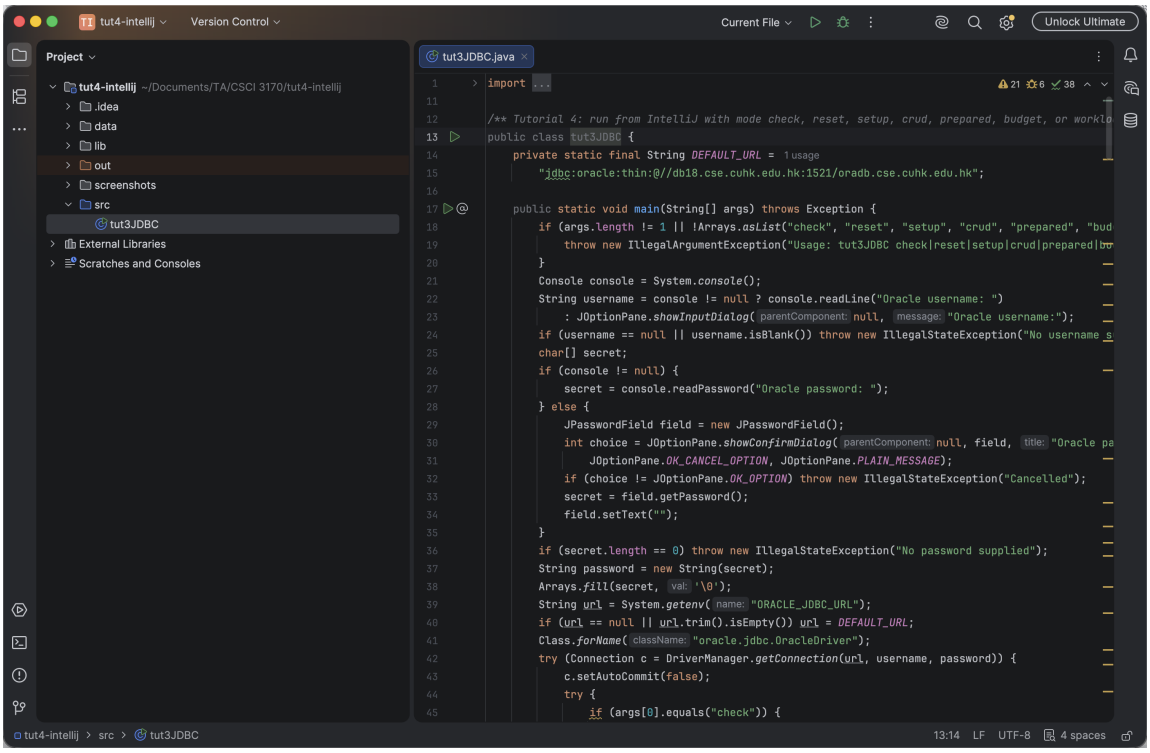
Preparation: project, access, and table state

Open the supplied `tut4-intellij/` folder. It contains `src/tut3JDBC.java`, `data/create_table.sql`, six `data/add*.sql` files, and `lib/ojdbc11.jar`. Use JDK 11 or later. Keep the working directory at `tut4-intellij/`, because `setup` reads `data/` by relative path. Use your *Oracle* account at the prompt; a CSE Linux login is only needed if you use the remote host or an SSH tunnel. Never place either password in source, commands, screenshots, or Git.

The machine must be able to reach `db18.cse.cuhk.edu.hk:1521`. If the connection times out, join the course network or approved VPN first. Test with `check` before changing any tables. The six lab tables are `OFFICE`, `ENGINEER`, `PROJECT`, `APPLICATION`, `MAINTAIN`, and `RESEARCH`. `reset` drops these tables and their contents. Save any work you need before using it; run it only in your own tutorial schema.

1. [Hands-on Practice] Create the project in IntelliJ IDEA

1. In IntelliJ IDEA, choose **File** → **Open** and select `tut4-intellij/`. Select JDK 11 or later for the module.
2. Confirm that `src/tut3JDBC.java`, `data/`, and `lib/ojdbc11.jar` appear in the Project pane.
3. Open **File** → **Project Structure** → **Modules** → **Dependencies**, press "+", select **JARs or Directories**, and add `lib/ojdbc11.jar`. Apply the change.



IntelliJ project structure.

Working directory

Relative paths start at the working directory. In Run/Debug Configurations, set it to the project root containing `data/`; otherwise the program may report `NoSuchFileException: data/...`.

Run configuration and Oracle URL

Choose **Edit Configurations**. Set Main class to `tut3JDBC`. In Program arguments, enter one mode at a time: `check`, `reset`, `setup`, `crud`, `prepared`, `budget`, `verify-budget`, `workload`, or `verify-workload`. Select the module containing the JDBC library and set the working directory to `tut4-intellij/`. The default JDBC URL is:

```
jdbc:oracle:thin:@//db18.cse.cuhk.edu.hk:1521/oradb.cse.cuhk.edu.hk
```

Code path. `main` selects a mode and opens one JDBC `Connection`. `check` queries `USER_TABLES` and counts each existing table. `reset` uses child-first `DROP TABLE` statements so foreign keys do not block parent tables. `initialize` reads `create_table.sql`, creates the six tables, adjusts `OFFICE.OADDRESS` to 50 characters, then runs the six `add*.sql` files. The simple semicolon splitter is intended only for these supplied SQL files. Each insert uses `executeUpdate`; `commit` saves the rows. Oracle DDL commits implicitly, so a failed setup may leave partial tables that must be reset before retrying.

2. [Reading Material] Transactions and JDBC resource lifetime

```
Class.forName("oracle.jdbc.OracleDriver");
try (Connection c = DriverManager.getConnection(url, user, password)) {
    c.setAutoCommit(false);
    try {
        // execute SQL here
        c.commit();
    } catch (Exception e) {
```

```
        try { c.rollback(); }
        catch (SQLException rollbackError) { e.addSuppressed(rollbackError); }
        throw e;
    }
}
```

What is a transaction?

A transaction is one logical unit of database work. For example, the `budget` mode may update eight PROJECT rows. The intended result is *all eight updates or none of them*; saving only the first few would leave an incomplete adjustment. A JDBC `Connection` is one database session. With `c.setAutoCommit(false)`, its INSERT, UPDATE, and DELETE statements remain in the current transaction until the program explicitly calls `commit()` or `rollback()`. Without that setting, JDBC normally commits each statement as soon as it succeeds.

Stage	This connection sees	A separate connection sees
After an UPDATE, before commit	Its changed row	The last committed row
After `commit()`	The changed row	The changed row
After `rollback()` instead	The original row	The original row

The separate connection cannot read the uncommitted change in Oracle's usual read-committed isolation. `executeUpdate()` tells us how many rows the statement changed *inside the transaction*; it does not mean the change is permanently saved. A later `rollback()` undoes those uncommitted changes. After `commit()`, a later rollback cannot undo that commit.

Why commit or roll back explicitly?

The code above commits only after every required statement succeeds. If one statement throws an exception, the catch block rolls back the work done in that transaction and rethrows the error so the caller can see it. The `crud` demonstration deliberately calls `rollback()`: students can observe the INSERT, UPDATE, and DELETE in one session without changing the seed data. The `budget` and `workload` modes call `commit()` so a new Java verification session can see their results.

Why try-with-resources?

`Connection`, `Statement`/`PreparedStatement`, and `ResultSet` hold database and driver resources. A `ResultSet` is a cursor over query rows; `next()` moves to the next row. `try (...)` closes resources automatically when the block ends, including when an exception occurs. Nested blocks close in reverse order: result set, statement, then connection. For example:

```
try (PreparedStatement p = c.prepareStatement(
        "SELECT PBUDGET FROM PROJECT WHERE PROJECTID = ?")) {
    p.setInt(1, 9); // Bind a value instead of joining it into SQL text.
    try (ResultSet r = p.executeQuery()) {
        if (r.next()) System.out.println(r.getInt("PBUDGET"));
    } // ResultSet closes here, even if reading failed.
} // PreparedStatement closes here.
```

Closing a resource is different from deciding the transaction outcome: use `commit()` or `rollback()` for that decision. `Statement` works for fixed SQL; `PreparedStatement` binds runtime values to `?` placeholders and can be reused safely. Oracle DDL such as `CREATE TABLE` commits implicitly, so rollback cannot undo a successful table creation. This is why `setup` may need a reset after a DDL failure.

3. [Reading Material] Review the CRUD workflow

Use OFFICE row 99 to demonstrate the full lifecycle. Check the affected-row count after each DML operation, then roll back so that the seed data stays unchanged.

Create / INSERT

```
try (PreparedStatement p = c.prepareStatement(
    "INSERT INTO OFFICE (OFFICEID, ONAME, OADDRESS, OPHONENUM) VALUES (?, ?, ?, ?)")) {
    p.setInt(1, 99); p.setString(2, "JDBC Demo Lab");
    p.setString(3, "Room 3170"); p.setString(4, "20000000");
    int n = p.executeUpdate();
    if (n != 1) throw new SQLException("INSERT affected " + n + " rows");
}
```

Read / SELECT

```
try (PreparedStatement p = c.prepareStatement(
    "SELECT OFFICEID, ONAME FROM OFFICE WHERE OFFICEID = ?")) {
    p.setInt(1, 99);
    try (ResultSet r = p.executeQuery()) {
        while (r.next()) System.out.println(r.getInt("OFFICEID") + " | " + r.getString("ONAME"));
    }
}
```

Update / UPDATE and Delete / DELETE

```
try (PreparedStatement p = c.prepareStatement(
    "UPDATE OFFICE SET ONAME = ?, OPHONENUM = ? WHERE OFFICEID = ?")) {
    p.setString(1, "JDBC Demo Lab Updated"); p.setString(2, "29999999"); p.setInt(3, 99);
    if (p.executeUpdate() != 1) throw new SQLException("UPDATE did not change one row");
}
try (PreparedStatement p = c.prepareStatement("DELETE FROM OFFICE WHERE OFFICEID = ?")) {
    p.setInt(1, 99); if (p.executeUpdate() != 1) throw new SQLException("DELETE did not change one
    row");
}
c.rollback(); // Do not save OFFICE 99 in this demo
```

The sequence is **baseline** → **INSERT** → **SELECT** → **UPDATE** → **SELECT** → **DELETE** → **SELECT** → **ROLLBACK**. After running `crud`, check `jdbc_crud.txt` for output.

4. Review the two applications from Tutorial 3

Both programs follow the same pattern: SELECT the source rows, calculate changes in Java, UPDATE with a condition on the old value, and commit after all updates succeed. The old-value condition detects a concurrent change to the value being updated.

Application A: project budget adjustment (`budget`)

Scenario. A project manager wants a program that flags overspending and proposes a revised budget. Use PROJECT from the supplied dataset.

Rule. If expenditure exceeds budget, set the new budget to expenditure plus 10%, rounded *up* to a multiple of 100. Otherwise keep the existing budget. This is a hypothetical policy we made up for this demonstration.

```java
static int newBudget(int oldBudget, int spent) {
    if (spent <= oldBudget) return oldBudget;
    return Math.toIntExact(((spent * 110L + 9999) / 10000) * 100);
}
```

1. Query `PROJECTID, PNAME, PBUDGET, PEXPENDITURE`.
2. Compute each new budget in Java and collect only changed projects.
3. Execute `UPDATE PROJECT SET PBUDGET=? WHERE PROJECTID=? AND PBUDGET=?`.
4. Require exactly one affected row per update; call `c.commit()` after all updates succeed.

For example, an old budget of 19,925 and expenditure of 29,325 produce a new budget of 32,300. Run with the `budget` argument. With the original data, expect these eight changes:

ID	Project	Expenditure	Old budget	New budget
9	Flume	27500	19838	30300
12	CoffeeScript	29325	19925	32300
13	ApacheDrill	28034	20115	30900
15	FalseCrypt	27515	20044	30300
20	GFS	27442	19888	30200
26	EC3	29427	20005	32400
27	Rapyuta	24622	19886	27100
29	MS-IIS	29424	20121	32400

IntelliJ walkthrough. After `setup`, open `src/tut3JDBC.java`. In **Run → Edit Configurations**, set Program arguments to `budget` and the working directory to `tut4-intellij/`; press Run. Enter the Oracle account when prompted. The Run window should list eight old-to-new budgets, then `Projects updated: 8` and `COMMITTED`. Change Program arguments to `verify-budget` and Run again.

Key code. The multiplier `110L` adds 10% using long arithmetic; `+9999` makes the integer division round up to the next hundred. `Math.toIntExact` rejects an overflow rather than silently truncating it. The program first collects changes while reading a `ResultSet`, then closes it and writes with one reused `PreparedStatement`:

```java
int next = newBudget(old, spent);
if (next != old) changes.add(new int[] {id, old, next});
// For each row: new budget, project ID, expected old budget.
p.setInt(1, row[2]); p.setInt(2, row[0]); p.setInt(3, row[1]);
requireOne(p.executeUpdate());
```

The `AND PBUDGET = ?` condition detects a changed source value; `requireOne` makes a zero-row update fail, leading to rollback. The `budget` branch commits only after the whole method succeeds. `verify-budget` opens a *new* JDBC connection, prints the eight example budgets, then lists any projects where expenditure still exceeds budget. Compare its budget values with the table above and expect `Projects still over budget: 0`. A second `budget` run should update zero projects. Verification is read-only and rolls back its session.

Application B: engineer workload reward (`workload`)

Rule: Count the distinct applications maintained by each engineer. Give no increase for fewer than three apps. Otherwise add 500 per app, capped at five apps (a maximum increase of 2,500).

```java
static int newSalary(int oldSalary, int apps) {
    if (apps < 3) return oldSalary;
    return Math.addExact(oldSalary, 500 * Math.min(apps, 5));
}
```

1. Query `MAINTAIN(ENGINEERID, APPID)` and use `Map<Integer, Set<Integer>>` to count distinct apps.
2. Read each engineer's name and original salary, then calculate the app count and new salary.
3. Execute `UPDATE ENGINEER SET ESALARY=? WHERE ENGINEERID=? AND ESALARY=?`.
4. Commit after every update succeeds; roll back if an exception occurs.

IntelliJ walkthrough. In the same Run configuration, change Program arguments to `workload`; press Run and enter the Oracle account. On the original data, expect three old-to-new salary lines, `Engineers updated: 3`, and `COMMITTED`. Then change the argument to `verify-workload` and Run again.

The first run on the original data should update these three engineers:

ID	Engineer	Apps	Old salary	Increase	New salary
10	Chan Ling Yun	6	28350	2500	30850
17	Ye Huanbang	4	35850	2000	37850
122	Jane Xiu	6	46100	2500	48600

Key code. The map groups rows by engineer; each `Set` removes duplicate application IDs. The salary function applies the three-app threshold and five-app cap. The update includes the old salary to detect a concurrent change:

```
appsByEngineer.computeIfAbsent(id, k -> new HashSet<>())
              .add(r.getInt("APPID"));
int count = appsByEngineer.getOrDefault(id, Collections.emptySet()).size();
int next = newSalary(old, count);
p.setInt(1, next); p.setInt(2, id); p.setInt(3, old);
requireOne(p.executeUpdate());
```

`verify-workload` uses Java JDBC queries on a new connection to print engineers with at least three distinct apps and the total salary. After the first committed run on the original data, expect the three rows above and `Total salary: 2734050`. This program awards the same increase again on every run. Run it once for the demonstration; to repeat it, reset and reload the original data first.

5. Run sequence and troubleshooting

1. Run `check`; for a fresh tutorial schema, run `reset`, then `setup`, then `check` again. Confirm the row counts: 8, 89, 14, 23, 26, and 71.
2. Run `prepared`. It queries OFFICE 1 and 7 by rebinding one prepared statement.
3. Run `crud`. Confirm that it writes `jdbc_crud.txt` and reports a rollback.
4. Run `budget`, then `verify-budget` in Java. Run `workload` once, then `verify-workload` in Java.

Symptom	What to check
`ClassNotFoundException: oracle.jdbc...`	Add `ojdbc11.jar` to Module Dependencies.
Connection timed out	Check access to `db18.cse.cuhk.edu.hk:1521`; join the course network or approved VPN.
`NoSuchFileException: data/...`	Set Working directory to the project root.
`ORA-01017`	Check the Oracle username and password; they may differ from CSE credentials.
`ORA-00942`	Check the account and the tables listed in `USER_TABLES`.
`ORA-00001`	A key already exists. Do not rerun setup without resetting the lab tables.

Practice exercises

1. Adapt the CRUD example to APPLICATION. Remember that `ADATE` is stored as an integer.
2. Add output to `budget` showing the old and new budgets and the expenditure ratio.
3. Explain how `workload` separates reading, calculation, and writing, and why an old-salary mismatch requires a rollback.
4. Write an INSERT-only exercise: insert OFFICE 99, commit, and query it from SQL*Plus. The original CRUD sequence deletes that row, so changing only its final rollback to commit will not retain it. Delete OFFICE 99 and commit after verification.